COPILOTTRAINING
TECHNICAL BRIEFING

VS Code Copilot through 1.139

Portable Infrastructure, Open Models, and a Closed Review Loop

BYOK chat without GitHub sign-in; Agents window access has separate requirements

CopilotTraining Tech Talk · 45–60 minutes

🤔 THE CORE QUESTION

"How does VS Code Copilot evolve from a window-bound chat to portable agent infrastructure?"

Releases through 1.139 change where agents run, which models they use, and how review happens.

Which capability is your team ready to act on tomorrow?

Developer

Which new models and local options can I use without changing my GitHub account?

Tech Lead

How do parallel sessions, worktrees, and remote hosts change how we assign agent work?

Platform Engineer

What controls exist for model gateways, MCP allowlists, and org-wide policy enforcement?

Open the Agents window

Dedicated agent-first surface — still missed by teams stuck in side chat

BYOK GA + Ollama

Signed-out chat works; desktop Agents window access is experimental

Vision paste + browser loop

Attach Excel/screenshots as images; GA browser tools with element comments

📖 TABLE OF CONTENTS

PART 1

Agent Infrastructure

AHP + SDK: sessions persist beyond the window

Decouple the session from the window; run agents on owned remote hosts

PART 2

Open Model Workbench

BYOK GA, Ollama, Custom Endpoint, utility models

Use BYOK chat without sign-in; check Agents window access separately

PART 3

Parallel Agent Work

Open the Agents window; multi-session + /btw

Awareness first — then multi-session workspace, side chats, live pills

PART 4

Closed-Loop Delivery

Vision paste + GA browser validation loop

Attach evidence, emulate, screenshot, annotate — without leaving the session

PART 1

Agent Infrastructure

The Copilot SDK and Agent Host Protocol detach the session from the window so agents run on infrastructure you own



Agent Host Protocol

Open spec: host owns state, clients sync and disconnect



Remote Machine Host

SSH or dev tunnel; session persists after VS Code closes



Copilot SDK Runtime

Copilot, Claude, Codex harnesses in one dedicated process

```
Preview: The session is no longer the window
↳ session persists on remote infra → reconnect on completion
```

Session Is No Longer the Window

Before AHP

Close VS Code → agent stops
Session lifetime = window lifetime

Context is lost on disconnect

One active session per open window

Long autonomous tasks compete with active work

With AHP (Preview)

Host persists after VS Code closes
Reconnect to a running session any time

Multi-client: same session, multiple windows

Long tasks run on infrastructure your team owns

Progress, tool outputs, and memory survive disconnect

Connect a Remote Agent Host in Three Steps

1

Open Agents Window

Go to the Remote tab inside the Agents window in VS Code stable preview

No separate tooling required

2

Connect via SSH or Tunnel

Use ~/.ssh/config, user@host, or a dev tunnel; select Use Dev Container for a configured project

Container: setting + Docker on remote host (gradual rollout)

3

Host Starts on Remote

VS Code installs its CLI server and starts the agent host on the target machine

Host persists after VS Code closes

Reconnect and verify the project toolchain

PART 2

Open Model Workbench

BYOK chat is GA without GitHub sign-in; signed-out Agents window use is experimental

🔑

BYOK Without Sign-In

Any provider key — GA, no GitHub account needed

🏠

Local via Ollama

Pull models locally; zero network requests, air-gap ready

🔧

Custom + Utility

Stable gateway endpoint + per-task model tuning

```
PRIMARY DEMO: model picker → Manage Models… → add provider
↳ models appear instantly — billing and rate limits via provider
```

BYOK Chat Is GA Without GitHub Sign-In

Cloud Providers

Anthropic, Azure OpenAI, Gemini, OpenAI, OpenRouter — add key in Manage Models…

- **Billing and rate limits via provider**
- **No Copilot quota consumed**

Local via Ollama

VS Code auto-discovers pulled Ollama models — zero network requests leave the machine

- **Air-gap and data-residency safe**
- **Full chat, tools, and MCP support**

Two Sign-In Boundaries

Inline/NES need GitHub sign-in; signed-out desktop Agents window use is experimental

- **Enable `allowSignedOutWhenUsable` + `byokModels` for Agent Host BYOK**
- **Browser Agents window always requires sign-in**

Local Models via Ollama: Zero Network Requests

terminal

```
# 1. Install Ollama (ollama.com)
# 2. Pull models locally
ollama pull llama3.2
ollama pull codestral

# 3. In VS Code:
# model picker → Manage Models… → Ollama
# VS Code discovers pulled models automatically
```

🔍 Auto-Discovery

VS Code finds all locally pulled Ollama models without manual registration

🔒 Air-Gap Ready

Zero network requests — data residency guaranteed for sensitive codebases

🛠️ Full Feature Parity

All chat features, tool execution, and MCP work with local Ollama models

Stable Custom Endpoint and Utility Model Tuning

settings.json

```
{
  // Route all VS Code Copilot chat through
  // your own gateway — Chat: Manage Language Models
  // Add → Custom Endpoint → choose API family

  // Split utility tasks to a cheaper model
  "chat.utilityModel": "[configured-model-id]",
  "chat.utilitySmallModel": "[fast-local-model-id]"
}
```

🌐 Custom Endpoint (Stable)

Any Chat Completions, Responses, or Messages API gateway — stable since v1.122

⚡ Utility Model Split

Frontier model for primary work; fast local model for titles, summaries, and commits

💰 Cost and Compliance

Route all chat traffic through your internal gateway for logging and cost control

Model Selection: Four Paths for Four Needs



Interactive Chat

Frontier model via Copilot or BYOK provider key



Large Codebase Analysis

BYOK provider with 1M-token context + direct API access



Utility Sub-Tasks

Fast local or low-cost model for titles, summaries, commit messages



Sensitive or Air-Gap

Ollama local model — zero data leaves the machine

PART 3

Parallel Agent Work

First open the Agents window — then multi-session work, side chats, and live activity become obvious



Open It

Title bar, Command Palette, or code -- agents



Multi-Session Workspace

Isolated worktrees, tools, and model config per session



/btw + Live Pills

Lateral questions and at-a-glance activity without polling

```
Most teams still only use side chat
↳ Open the Agents window before teaching parallel features
```

Open the Agents Window First

📍

Title Bar

Select Open in Agents in the VS Code title bar — always one click away

⌨️

Command Palette

Chat: Open Agents Window via Ctrl+Shift+P / ⇧⌘P

💻

CLI

code --agents from the terminal — scriptable entry point

🌐

Browser / Welcome

Welcome-page link or insiders.vscode.dev/agents

Desktop sign-in is required by default; signed-out Agents window access is experimental and setting-gated

Agents Window: From Side Chat to Workspace

Side Chat Only (common habit)

- One conversation buried in the editor sidebar
- No multi-workspace sessions list
- Hard to track concurrent agent work
- Changes and files live in other panels

Agents Window (Preview workspace)

- Dedicated window: sessions across workspaces
- Chat + Changes + Files for the active session
- Multiple sessions side-by-side with isolated worktrees
- /btw side chats and live activity pills

/btw Side Chats and Live Activity Pills

/btw Side Chat (GA v1.132)

Opens without pausing the active turn
Main agent keeps executing while you type

Shares context and prompt cache with main chat

Captures observations without interrupting the task

/btw is the builder pattern a good fit here?
Ask mid-tool-chain; main turn keeps running

Live Activity Pills (GA v1.132)

Changes, Previews, Subagents, Browsers
All visible at a glance in the Agents window

Navigate directly to an active subagent or browser

Pills clear when a turn completes — no noise accumulation

Replaces manual polling for Autopilot sessions
No need to check the terminal and come back

PART 4

Closed-Loop Delivery

Vision paste for everyday evidence — then GA browser tools close the build-validate-review loop

Copilot Vision (GA)

Paste Excel, mockups, PDFs — attach evidence, not walls of text

GA Browser Tools

Navigation, screenshot, interaction — on by default

Element Comments

Select elements in v1.132, anchor a comment per element

```
CLIMAX DEMO: implement → emulate → screenshot → annotate elements → re-validate
↳ Vision paste for human evidence; browser tools for agent validation
```

Paste Evidence Instead of Walls of Text

🖼️ Copilot Vision (GA v1.128)

Paste, drag-drop, or attach images and PDFs
Ask, plan, and agent modes

Available on Free, Pro, Pro+, Business, and Enterprise

Excel range → image attachment
Clipboard image data attaches as vision context

Same path for mockups, error dialogs, whiteboards

🌐 Browser Add-to-Chat

Add Screenshot to Chat
Capture the current viewport as an image

Add Element to Chat
Selected elements with styles and screenshots

Add Console Logs to Chat
Runtime output without copy-paste sprawl

You already have the page — attach it and ask

Browser Validation: Before vs After GA Tools

Closing the loop without leaving the session

❌ Manual Browser Validation

1. **Code the UI change**
   Edit in the editor
2. **Open browser manually**
   Switch to a separate browser tab
3. **Resize manually for mobile**
   Approximate the device viewport
4. **Annotate in external tool**
   Screenshot, paste, mark up separately
5. **Copy feedback back into chat**
   Describe the issues in words

🔄 3–5 context switches per iteration

✅ With GA Browser Tools

1. **Code the UI change**
   Edit in the editor
2. **Agent opens localhost page**
   Browser opens in the integrated panel
3. **Agent emulates device preset**
   iPhone 15, Galaxy S24, or custom viewport
4. **Screenshot attaches to chat turn**
   Validation evidence is part of the conversation
5. **Select elements, anchor a comment each**
   Precise per-element feedback (v1.132)

✅ Full loop inside one session

3–5 context switches per iteration → 0

The Closing Demo: Emulate, Screenshot, Annotate

Implement → emulate → screenshot → element comment → re-validate, all in one session

```
$ agent
> "Implement the mobile login form and validate it on iPhone 15"
Copilot agent — browser tools active:
  Opening http://localhost:3000/login ...
  Emulating iPhone 15 (390x844 viewport) ...
✅ Screenshot attached to this turn: login-mobile.png
> "[element comment on .btn-submit] Tap area is 28px — needs 44x44px min for accessibility"
✅ Applied: min-height: 44px; padding: 12px on .btn-submit
✅ Screenshot attached: login-mobile-v2.png — tap area confirmed
```

implement → emulate → screenshot → annotate elements → re-validate: all in one session

Review and PR Handoffs

📋

In-Window Diff Review

Apply, revert, or cherry-pick per file; inline comments the agent acts on in later turns

Preview in Agents window

🔄

CI Feedback in Session

Failed check details surface in the session — read, fix, and re-run without tab-switching

Preview: CI response in-window

💬

PR Handoff + Feedback

Review the generated PR title, description, and draft choice; create directly or ask the agent

Agent Merge: experimental, opt-in

PR feedback: preview in-session

From Window-Bound Sessions to Portable Agent Infrastructure

Before

- Agents only used from the side Chat view
- One provider; GitHub sign-in required for chat and tools
- Paste walls of Excel/text instead of visual evidence
- Manual browser validation with context switches per iteration

After

- Agents window open as the multi-session agent surface
- BYOK chat or local Ollama without GitHub sign-in
- Vision paste for Excel/mockups; browser Add Screenshot to Chat
- GA browser loop with device emulation and element comments

Open

Agents window — title bar, palette, or code --agents

GA

Vision paste + browser tools with device emulation

Check

Signed-out Agents window is experimental; browser requires sign-in

✅ WHAT YOU CAN DO TODAY

Today

- Open the Agents window (title bar, Command Palette, or code --agents)
- Paste an Excel range or screenshot into Chat — Vision attaches it as an image
- Open model picker → Manage Models… → add a BYOK or Ollama provider

This Week

- Try /btw during a long agent turn; watch live activity pills
- Use browser Add Screenshot / Element to Chat on a localhost page
- Run a UI task and validate it with GA browser device emulation

This Month

- Evaluate Custom Endpoint for team-wide cost and compliance routing
- Run parallel Agents window sessions for independent concurrent tasks
- Create a PR from an Agent Host session; review diffs and CI feedback in-window

🔑 Key Takeaway

Check access and toolchain, open the Agents window, then validate with vision + browser evidence.

REFERENCES

📖 Official Documentation

VS Code 1.139: remote Dev Container agents
SSH, Tunnel, WSL; gradual rollout and remote Docker prerequisite

VS Code 1.138: create PR from agent session
Review PR form; optional Agent Merge is experimental

VS Code release notes: August 5, 2026 (v1.132)
Element comments, /btw, live pills, multi-window agent sessions

Use the Agents window (Preview)
Desktop signed-out settings and browser sign-in boundary

Copilot vision is generally available
Supported formats and plan availability for image/PDF attachments

🛠️ Related Content

Copilot SDK Talk
Deep dive into the Copilot SDK harness and Agent Host Protocol

MCP Apps Talk
Connecting MCP servers to VS Code Copilot sessions and workflows

VS Code Copilot through 1.139

PORTABLE AGENT INFRASTRUCTURE, OPEN MODELS, AND A CLOSED REVIEW LOOP

Agents Window

Open it — multi-session agent surface most teams still skip

BYOK GA

Chat without sign-in; Agents window signed-out use is experimental

Vision GA

Paste Excel/screenshots/PDFs instead of walls of text

Browser GA

Emulate, screenshot, element comments — no context switch

Which changes your team first — opening the Agents window, model choice, or vision + browser evidence?